JAVA DOCS

Java Concurrency in Practice — Deep Dive

Thread Safety, the Java Memory Model, and Building Correct Concurrent Programs

Java Agentic AI — Knowledge Base Reference

1. What Does 'Thread-Safe' Actually Mean?

A class is thread-safe if it behaves correctly when accessed from multiple threads, regardless of the scheduling or interleaving of their execution by the runtime, with no additional synchronization required by the calling code. Correctness under concurrency is fundamentally about managing access to **shared mutable state** — if state is never shared, or never mutated, there's nothing to protect.

Strategy	Description
Don't share the state	Confine mutable state to a single thread (thread confinement, e.g. ThreadLocal)
Make the state immutable	Immutable objects require no synchronization since they can't change
Synchronize access	Use locks/atomics to serialize access to genuinely shared mutable state

2. Race Conditions

A race condition occurs when the correctness of a computation depends on the relative timing or interleaving of multiple threads. The most common form is a **check-then-act** race: a thread observes some state, then acts based on it, but the state can change between the observation and the action.

```
public class UnsafeCounter {
    private int count = 0;
    public void increment() {
        count++;    // NOT atomic: read count, add 1, write count back - 3 separate steps
    }
}
// If two threads call increment() concurrently, both can read the same value before either
// writes back, and one increment is silently lost.
```

```
// Lazy initialization race - classic check-then-act bug
private ExpensiveObject instance;
public ExpensiveObject getInstance() {
    if (instance == null) {             // check
        instance = new ExpensiveObject(); // act - two threads can both pass the check
    }
    return instance;
}
```

3. The Java Memory Model (JMM)

The JMM defines what values a read of shared memory is allowed to see given the writes that occurred, in the presence of compiler optimizations, CPU caches, and instruction reordering. Without proper synchronization, one thread's writes are not guaranteed to ever become visible to another thread — this is a real, not theoretical, concern on modern multi-core hardware with per-core caches.

3.1 Happens-Before Relationships

The JMM guarantees visibility and ordering only across specific 'happens-before' edges. If action A happens-before action B, then A's effects are guaranteed visible to B. Key sources of happens-before edges:

- Each action in a thread happens-before every subsequent action in that same thread (program order).
- An unlock on a monitor happens-before every subsequent lock on that same monitor (by any thread).
- A write to a volatile field happens-before every subsequent read of that same field.
- A call to Thread.start() happens-before any action in the started thread.
- All actions in a thread happen-before any other thread successfully returns from a join() on it.

Note: *Without one of these established relationships, the compiler and CPU are free to reorder, cache, or otherwise not propagate a write — which is why 'it worked in my testing' is not proof of correctness for concurrent code lacking proper synchronization; visibility bugs are often non-deterministic and hardware/JIT-dependent.*

4. Intrinsic Locks (synchronized)

```
public class SafeCounter {
    private int count = 0;
    public synchronized void increment() { // acquires the monitor of 'this'
        count++;
    }
    public synchronized int getCount() {     // MUST also synchronize the read
        return count;
    }
}
```

Note: *A very common mistake: synchronizing the writer method but not the reader. Without synchronizing getCount() too, there's no happens-before edge guaranteeing the reading thread sees the latest value — the read can return a stale, cached value indefinitely on some JVM/hardware combinations.*

5. Guarding State with Locks — Consistently

Every mutable field that's shared across threads must be guarded by the SAME lock for every access, read or write. Using different locks for different accesses to the same field provides no real protection at all — it's a common but completely ineffective pattern sometimes called 'lock splitting done wrong.'

```
public class BankAccount {
    private final Object lock = new Object();
    private double balance;

    public void deposit(double amount) {
        synchronized (lock) { balance += amount; }
    }
    public void withdraw(double amount) {
        synchronized (lock) {                    // SAME lock object every time
            if (amount > balance) throw new IllegalStateException("Insufficient funds");
            balance -= amount;
        }
    }
    public double getBalance() {
        synchronized (lock) { return balance; }    // reads must be guarded too
    }
}
```

6. Atomic Variables

java.util.concurrent.atomic classes (AtomicInteger, AtomicLong, AtomicReference) provide lock-free thread safety for single variables using CAS (Compare-And-Swap) hardware instructions — far cheaper than acquiring a lock under moderate contention, since there's no thread blocking or context switch involved.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();                    // atomic read-modify-write

// CAS loop pattern (what incrementAndGet does internally)
int oldValue, newValue;
do {
    oldValue = counter.get();
    newValue = oldValue + 1;
} while (!counter.compareAndSet(oldValue, newValue));
```

Note: *CAS-based updates can spin and retry under high contention (many threads competing for the same variable), but avoid the overhead of OS-level thread blocking that a traditional lock incurs, making atomics excellent for simple counters and flags under moderate contention.*

7. Concurrent Collections

Class	Key property
ConcurrentHashMap	Fine-grained internal locking/CAS — far higher throughput than a synchronized HashMap und
CopyOnWriteArrayList	Every write copies the entire backing array; ideal for read-heavy, rarely-written lists (e.g. listen
BlockingQueue implementations	put()/take() block when full/empty — the foundation of producer-consumer pipelines

```
// Producer-consumer with a BlockingQueue - no manual wait/notify needed
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100);

// Producer thread
queue.put(task);     // blocks if the queue is full

// Consumer thread
Task task = queue.take(); // blocks if the queue is empty
```

8. Building Blocks: Latches, Barriers, Semaphores

```
// CountDownLatch - wait for N events to complete before proceeding
CountDownLatch latch = new CountDownLatch(3);
for (int i = 0; i < 3; i++) {
    executor.submit(() -> {
        doWork();
        latch.countDown();
    });
}
latch.await();     // blocks until the count reaches zero

// Semaphore - limit concurrent access to a resource pool
```

```
Semaphore semaphore = new Semaphore(5); // 5 permits - e.g. max 5 concurrent DB connections
semaphore.acquire();
try {
    useResource();
} finally {
    semaphore.release();
}

// CyclicBarrier - wait for all N threads to reach a common point, then proceed together
CyclicBarrier barrier = new CyclicBarrier(4, () -> System.out.println("All threads
ready"));
```

9. Avoiding Liveness Hazards

Hazard	Description	Mitigation
Deadlock	Circular wait on locks held by different threads	Always acquire locks in a consistent global order; use tryL
Starvation	A thread is perpetually denied access to a resource	Use fair locks (ReentrantLock(true)) if strict fairness matte
Livelock	Threads keep changing state in response to each other without making progress	Introduce randomized backoff before retrying

10. Designing Thread-Safe Classes

- Identify exactly what variables constitute the object's state and what invariants constrain them (e.g. 0 <= count <= capacity).
- Establish a synchronization policy: which lock guards which state, and enforce it consistently everywhere that state is touched.
- Prefer confining mutable state to a single method/thread over sharing it — the simplest concurrency bug is the one that can't happen because nothing is actually shared.
- Document your class's thread-safety guarantees explicitly (thread-safe, conditionally thread-safe, not thread-safe) — future maintainers need this and can't safely infer it from reading the code.

11. Quick Interview Recap

- Why isn't count++ thread-safe even though it's one line of Java? It compiles to three separate steps (read, increment, write) that can interleave between threads without synchronization.
- What does volatile guarantee, and what does it NOT guarantee? It guarantees visibility (reads see the latest write) and prevents certain reordering, but does NOT make compound read-modify-write operations atomic.
- Why must both readers and writers of shared state synchronize on the same lock? Because the happens-before relationship (and thus guaranteed visibility) is only established between an unlock and a subsequent lock ON THE SAME MONITOR — synchronizing writes but not reads (or using different locks) provides no visibility guarantee at all.

- When would you choose an AtomicInteger over synchronized for a counter? When contention is moderate and the operation is a simple read-modify-write — CAS avoids the cost of OS-level thread blocking that acquiring a lock entails.
- What's the difference between deadlock, livelock, and starvation? Deadlock: threads permanently blocked waiting on each other. Livelock: threads keep actively responding to each other without making progress. Starvation: a thread is repeatedly denied access to a resource it needs, even though the system as a whole is progressing.